

Python

Conditions

```
if condition :  
    ...  
elif ... :  
    ...  
else :  
    ...
```

Fonctions

```
def f(a,b=0,c=0):  
    instruction  
    return ...  
  
type(var) retourne le type de  
la variable.
```

Boucles

```
while ... :  
    ...  
  
for x in sequence :  
    ...  
  
for x in range(2,7)  
range(7) → 0 à 6  
range(2,7,2)  
→ 2 à 6 step by 2  
  
enumerate(list)  
→ sequence of tuples  
(index, element)
```

Division

```
/ division  
// division entière  
% reste
```

Booléens

```
and  
or
```

Chaînes de caractères

```
"simple quote"  
'double quote'  
"""triple double quote"""  
"""docs"""
```

Lists

```
L = []  
L = [1, "txt"]  
  
list.c = [1]*10  
list(range(5))  
0 à 4  
  
— len(list)  
— min(list)  
— max(list)  
  
if "..." in list :  
    ...  
else :  
    ...  
  
Méthodes :  
— len(list)  
— list.append(x)  
— list.insert(i,x)  
— list.extend(list2)  
— list.pop()  
— list.pop(i)  
— list.remove(x)
```

Extraction d'éléments

```
list[i :start :j-end excluded]  
list[i :j :end excluded]  
list[i :start :]  
list[i :j :step-size]
```

On peut aussi faire :
`a = ["1",2,"("]`

Dictionnaires

```
dict = { ... : ... }  
dict["..."] = ...  
  
.keys() .values()  
.pop(key)
```

Tuples

Similaire à lists mais fixed in size and immutable.

```
tuple = (...,...)
```

On ne peut pas modifier.
Je peux typer une list sans typer en tuple.

Fonctions utiles

```
obj = max(...)  
return ..., ...  
  
a,b = min,max(...)
```

List comprehension

```
[thing for thing in list]
```

Ex :

```
listA = [1,2,3,4]
```

```
listB = [2*elem for elem in listA]
```

```
[thing for thing in list if condition]
```

NumPy

```
import numpy as np
```

Creating arrays

Convert normal python arrays :

```
np.array([...])
```

Using built-in functions :

```
np.zeros((2,3))
```

```
np.ones(...)
```

```
np.eye(4)
```

```
np.arange(5,11)
```

```
np.ones like(...)
```

```
np.array([1,0], dtype=np.int32)
```

```
astype(np.float32)
```

```
.ndim
```

```
.size
```

```
.shape
```

```
.dtype
```

Get elements :

```
x[2 : :2, : :2]
```

```
np.concatenate([arr1,arr2], axis=0)
```

```
np.stack()
```

```
np.sum()
```

```
np.min np.max np.argmax
```

```
np.random.rand(3,2)
```

Matplotlib

```
import numpy
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

fig = plt.figure() → creates a figure

```
plt.plot(x,y, marker='*', color='r')
```

```
plt.xlabel("x")
```

```
plt.ylabel("y")
```

```
plt.title("title")
```

```
plt.show() → pour afficher
```

```
plt.close(fig) → close the plot
```

On peut avoir plusieurs fonctions sur le graph en faisant plusieurs plt.plot(...)

plt.scatter(...) → même chose que plot mais sans les lignes.

```
img = plt.imread("img/...jpg")
```

→ reads an image

```
plt.imshow(img)
```

Masking :

```
mask = x > 20
```

```
x[mask]
```